TandemPay Database Guide 🗄️

This guide covers everything you need to know about accessing, viewing, and manipulating the TandemPay database both in your local development environment and in your live production environment.

---

1. Local Development (SQLite)

In your local environment, TandemPay uses a lightweight file-based database.

- **Database File:** `backend/splitease.db`
- **Engine:** SQLite (accessed via `aiosqlite` in FastAPI)

How to View & Edit Data Locally

Because it's a simple `.db` file, you can't access it via a web URL. Instead, you need a local database viewer.

Option A: VS Code Extension (Recommended)

1. Open VS Code extensions and search for **"SQLite Viewer"** (by Florian Klampfer).
2. Install it.
3. In your VS Code file explorer, click on `backend/splitease.db`. It will open in a beautiful grid inside VS Code where you can view tables, edit rows, and run custom queries instantly.

Option B: Standalone App

1. Download DB Browser for SQLite.
2. Open the app, click "Open Database", and select your `splitease.db` file.
3. You can browse data, execute SQL, and modify structures.

---

2. Production (Render & PostgreSQL)

In production, Tandem uses a robust, high-performance PostgreSQL database hosted on Render.

- **Engine:** PostgreSQL
- **Location:** Hosted on Render.com under your account.

How to Access Production Data

Because this database is live and secure, you connect to it using "External Database URLs" or Render's built-in dashboard.

Option A: Using pgAdmin or DBeaver (Desktop GUI) This is the best way to handle data in production.

1. Download a database GUI like DBeaver (Free) or pgAdmin.
2. Go to your Render Dashboard -> Select your PostgreSQL Database instance.
3. Find the **"External Database URL"** (it looks like `postgres://user:password@external-host.render.com/dbname`).
4. In DBeaver/pgAdmin, create a new "PostgreSQL" connection, paste the URL, and connect. You now have full access to view, edit, and delete production data.

> *[!WARNING] Be extremely careful when using DBeaver connected to your External Render Database. Any changes made here are live and affect real users immediately.*

Option B: Render psql Console If you just need to run a quick query:

1. Go to your Render Dashboard -> PostgreSQL instance.
2. Look for the "Connect" button or "Shell" access. It will give you a `psql` command.
3. You can run raw SQL queries like: `SELECT * FROM users WHERE email = 'test@example.com';`

---

3. Key Data Handling Commands & Scripts

Most database modifications should actually be done through the backend code or scripts (using `SQLAlchemy`) so you don't accidentally corrupt relationships (like deleting a user but leaving their expenses, which crashes the app).

Interacting via Python (The Safe Way)

If you need to do bulk operations (like the password reset issue we fixed previously), write a quick Python script explicitly defining the database session.

Create a file named `db_script.py` in your backend folder with the following template:

```python
import asyncio
from app.database import SessionLocal
from app.models import User, Expense, Group

async def fix_data():
    async with SessionLocal() as db:
        # Example 1: Find a specific user
        # user = await db.execute(select(User).filter(User.email == "target@email.com"))
        # user = user.scalar_one_or_none()

        # Example 2: Update a value
        # if user:
        #     user.is_active = True
        #     await db.commit()

        print("Done!")

if __name__ == "__main__":
    asyncio.run(fix_data())
```

Run it via: `python db_script.py` (Local) or open a Render Shell and run it in production.

Schema Changes (Alembic)

If you add new fields to your models in `app/models.py` (e.g., adding `phone_number` to `User`), you **must** generate a migration so the database structure updates.

1. **Create the Migration:** `alembic revision --autogenerate -m "added phone number"`
2. **Apply the Migration (Local):** `alembic upgrade head`
3. **Apply the Migration (Production):** This usually happens automatically when Render builds the app (assuming you added `alembic upgrade head` to your `build.sh` file).

Technical Summary

- **Local URL:** `sqlite+aiosqlite:///./splitease.db`

- **Prod URL Structure:** `postgresql+asyncpg://user:password@host/dbname`
- **ORM:** SQLAlchemy 2.0 (Async)
- **Migrations:** Alembic